LegalLens – Contract Clause & Compliance Analyzer
Table of Contents

Project Overview

LegalLens is a role-based legal document management and compliance analysis platform that orchestrates contract upload, automated clause detection, compliance scoring, and reviewer collaboration natively. It provides structural entry points for uploading legal documents (PDF/text), running rule-based clause analysis, performing compliance checks against configurable rules, and managing clause templates while rigorously encapsulating business logic for post-upload automated analysis, risk-level scoring, and immutable audit trail recording securely.

The system enforces role-based access control dynamically allocating distinct application privileges for ROLE_ADMIN, ROLE_LEGAL_REVIEWER, and ROLE_CLIENT roles inherently. All interactions strictly utilise a stateless filter topology that executes the standard HS256 JWT algorithm implicitly signing JSON Web Tokens to prevent unauthorised record modifications and natively restricted administrative operations internally safely.

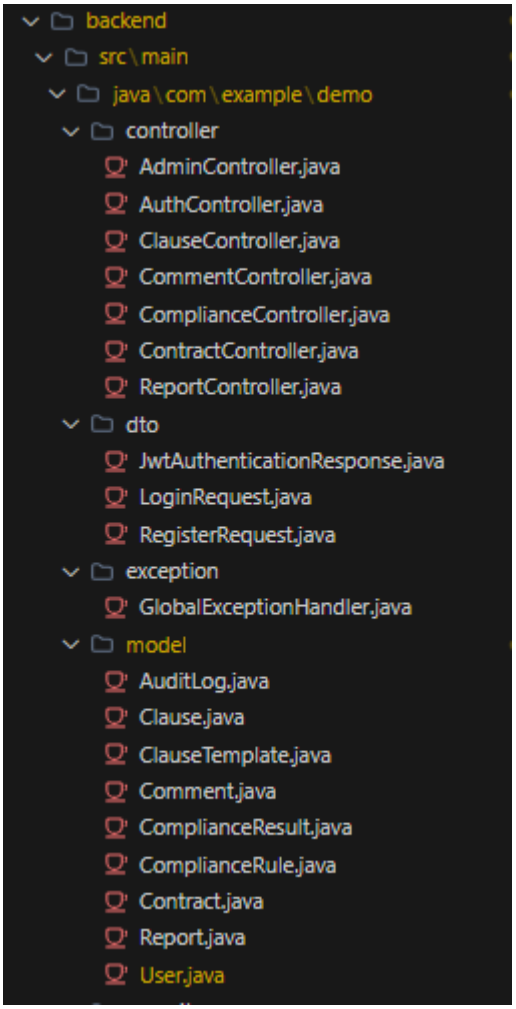
Technology Stack

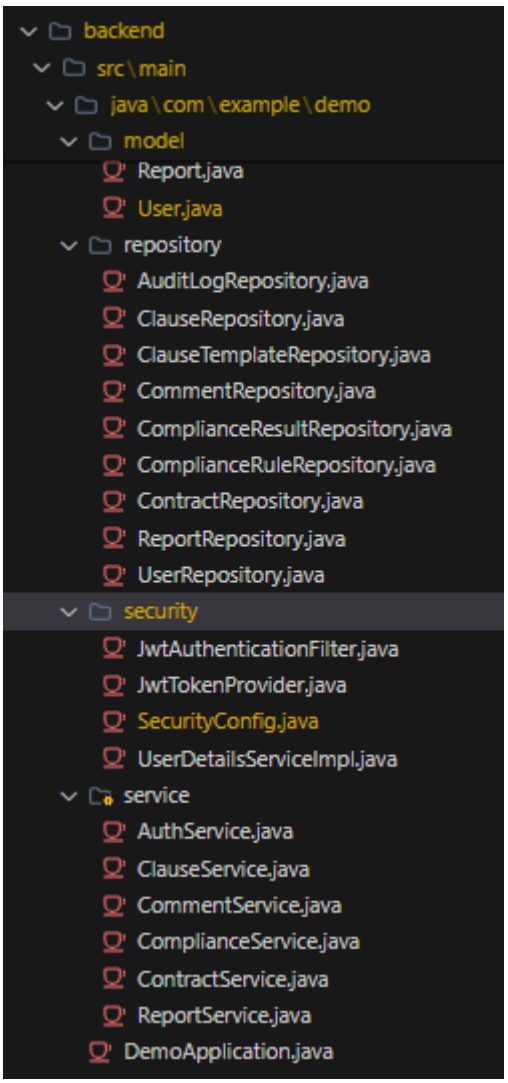
- **Framework:** Spring Boot
- **Application Name:** legallens-backend
- **Java Version:** Java 17+
- **Build Tool:** Maven
- **Frontend:** React (with Ant Design component library)
- **Database:** MySQL
- **ORM:** Spring Data JPA (Hibernate)
- **Security:** Spring Security with @EnableMethodSecurity
- **Password Encoding:** BCrypt
- **JWT:** JJWT library (HS256)

- **PDF Parsing:** Apache PDFBox (Loader.loadPDF, PDFTextStripper)
- **File Storage:** Local filesystem (./uploads directory, UUID-prefixed filenames)

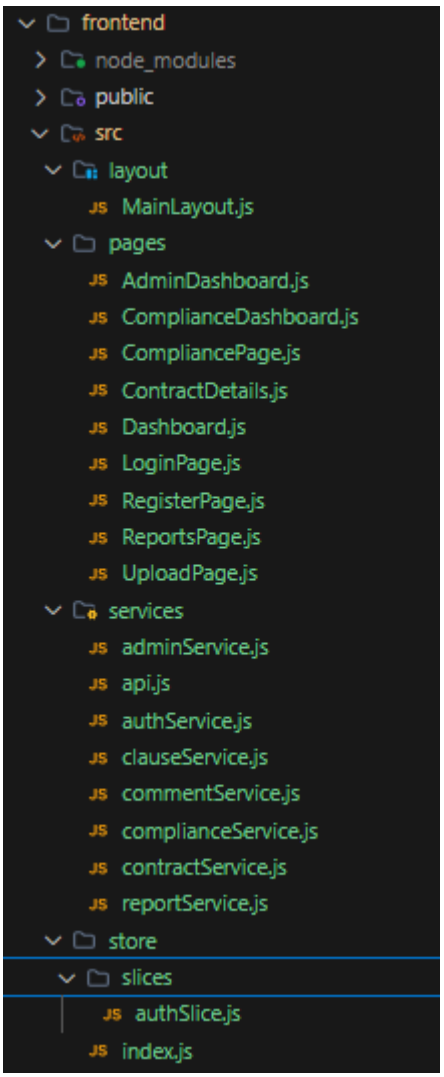
Project Structure

Backend (com.example.demo**):**





Frontend (src/**):**



Dependencies and Configuration

Backend:

- Spring Boot Starter Web
- Spring Boot Starter Data JPA
- Spring Boot Starter Security
- Spring Boot Starter Validation
- MySQL Connector/J
- JWT Library (jjwt-api, jjwt-impl, jjwt-jackson)
- Apache PDFBox (for PDF text extraction)
- Lombok
- Spring Boot Starter Test
- TestNG (for ProjectValidationTests)

Frontend:

- React, React DOM
- React Router DOM (BrowserRouter, Routes, Route, Navigate, useParams, useNavigate, useLocation, Link)
- Redux Toolkit (@reduxjs/toolkit)
- React Redux (Provider, useSelector, useDispatch)

- Axios
- Ant Design (antd) — Form, Input, Button, Card, Typography, message, Layout, Menu, Sider, Table, Select, DatePicker, InputNumber, Upload, Tag, Tabs, Progress, Modal, Spin etc.
- Ant Design Icons (@ant-design/icons)
- Testing Library (Jest DOM, React, User Event)

Database Configuration

- **Database Name:** legallens_db
- **URL:** jdbc:mysql://localhost:3306/legallens_db?createDatabaseIfNotExist=true&useSSL=false&allowPublicKeyRetrieval=true
- **Driver:** com.mysql.cj.jdbc.Driver
- **DDL Auto:** update
- **Dialect:** org.hibernate.dialect.MySQLDialect
- **Server Port:** 8080
- **Multipart Max File Size:** 10MB
- **Multipart Max Request Size:** 10MB

Custom Properties:

- app.upload.dir=./uploads — filesystem directory for uploaded contract files
- app.jwt.secret=404E635266556A586E3272357538782F413F4428472B4B6250645367566B5970 — HS256 signing key
- app.jwt.expiration=86400000 — token expiry in ms (24 hours)

Entity Models
1. User Entity (Table: users)

Fields: id (Long), username (String), password (String), role (String), fullName (String), email (String)

Validation:

- username: @Column(unique=true, nullable=false)
- password: @Column(nullable=false)
- role: @Column(nullable=false) — values: ROLE_ADMIN, ROLE_LEGAL_REVIEWER, ROLE_CLIENT
- email: @Column(unique=true, nullable=false)

Note: Includes a convenience constructor User(username, password, role, fullName, email).

2. Contract Entity (Table: contracts)

This is the **primary entity**. Must be annotated with @Entity and @Table(name="contracts") . Must have at least **4 declared fields** . Must have a field named contractName . Must have a field uploadedBy with @JoinColumn whose name contains an underscore .

Fields: id (Long), contractName (String), contractType (String), description (String), firstParty (String), secondParty (String), startDate (LocalDate), endDate (LocalDate), contractValue (Double), jurisdiction (String), fileUrl (String), uploadDate (LocalDateTime), status (String), complianceScore (Integer), uploadedBy (ManyToOne → User)

Validation:

- description: @Column(columnDefinition="TEXT")
- uploadedBy: @ManyToOne, @JoinColumn(name="uploaded_by") — the name value "uploaded_by" contains "_"

Note: Default constructor sets uploadDate = LocalDateTime.now() and status = "UPLOADED". Status values: UPLOADED, UNDER_REVIEW, APPROVED, REJECTED.

3. Clause Entity (Table: clauses)

Fields: id (Long), clauseType (String), content (String), riskLevel (String), contract (ManyToOne → Contract)

Validation:

- content: @Column(columnDefinition="TEXT")
- contract: @ManyToOne, @JoinColumn(name="contract_id")

Note: clauseType values used by automated analysis: "Payment Terms", "Confidentiality", "Termination", "Limitation of Liability", "Renewal". riskLevel values: "LOW", "MEDIUM", "HIGH". Risk overrides: content containing "unlimited liability" → HIGH; content containing "penalty" → MEDIUM.

4. ClauseTemplate Entity (Table: clause_templates)

Fields: id (Long), name (String), category (String), defaultContent (String), recommendedRiskLevel (String)

Validation:

- defaultContent: @Column(columnDefinition="TEXT")

Note: Used by admins to create standard clause templates for reference. Category examples: Payment, Liability.

5. ComplianceRule Entity (Table: compliance_rules)

Fields: id (Long), ruleName (String), description (String), severityLevel (String)

Validation:

- description: @Column(columnDefinition="TEXT")

Note: severityLevel values: LOW, MEDIUM, HIGH. Managed exclusively by ROLE_ADMIN.

6. ComplianceResult Entity (Table: compliance_results)

Fields: id (Long), complianceScore (Integer), issuesFound (String), analyzedAt (LocalDateTime), contract (ManyToOne → Contract)

Validation:

- issuesFound: @Column(columnDefinition="TEXT")
- contract: @ManyToOne, @JoinColumn(name="contract_id")

Note: Default constructor sets analyzedAt = LocalDateTime.now(). Compliance score formula: 100 - (highRisk × 15 + mediumRisk × 10 + lowRisk × 5), minimum 0. Issues string lists high/medium risk clause counts and MISSING required clause types.

7. Comment Entity (Table: comments)

Fields: id (Long), content (String), createdAt (LocalDateTime), reviewer (ManyToOne → User), contract (ManyToOne → Contract)

Validation:

- content: @Column(columnDefinition="TEXT")
- reviewer: @ManyToOne, @JoinColumn(name="reviewer_id")
- contract: @ManyToOne, @JoinColumn(name="contract_id")

Note: Default constructor sets createdAt = LocalDateTime.now().

8. AuditLog Entity (Table: audit_logs)

Fields: id (Long), action (String), username (String), details (String), logTimestamp (LocalDateTime)

Validation:

- logTimestamp: @Column(name="log_timestamp")

Note: Default constructor sets logTimestamp = LocalDateTime.now(). Convenience constructor AuditLog(action, username, details) calls this() first. Action values logged by system: "LOGIN", "REGISTER", "CONTRACT_UPLOAD". Ordered by logTimestamp DESC.

9. Report Entity (Table: reports)

Fields: id (Long), reportType (String), generatedDate (LocalDateTime), summary (String)

Validation:

- summary: @Column(columnDefinition="TEXT")

Note: Default constructor sets generatedDate = LocalDateTime.now().

Repositories

All repositories must be interfaces extending JpaRepository<Entity, Long>.

1. UserRepository

- Optional<User> findByUsername(String username)
- Boolean existsByUsername(String username)
- Boolean existsByEmail(String email)

2. ContractRepository (Primary Repository)

Must be an interface extending JpaRepository . Must have at least one findBy-prefixed method .

- List<Contract> findByUploadedBy(User user)

3. ClauseRepository

- List<Clause> findByContract(Contract contract)

4. ClauseTemplateRepository

Extends JpaRepository<ClauseTemplate, Long> with no custom methods (uses standard findAll(), findById(), save(), deleteById()).

5. ComplianceResultRepository

- Optional<ComplianceResult> findByContract(Contract contract)

6. ComplianceRuleRepository

Extends JpaRepository<ComplianceRule, Long> with no custom methods.

7. CommentRepository

- List<Comment> findByContract(Contract contract)

8. AuditLogRepository

Must have a method name containing "OrderBy".

- List<AuditLog> findAllByOrderByLogTimestampDesc()

9. ReportRepository

Extends JpaRepository<Report, Long> with no custom methods.

Service Layer

Services manage business logic between entities and DTOs natively.

1. ContractService

Annotated with @Service and @Transactional. Must have at least **3 declared methods** . Must have at least one method whose name (lowercased) contains "contract" . At least one method must return a non-void type.

- Contract uploadContract(MultipartFile file, String username, String contractName, String contractType, String description, String firstParty, String secondParty, String startDate, String endDate, Double contractValue, String jurisdiction) throws IOException — resolves User by username, creates upload directory if absent, stores file with UUID.randomUUID().toString() + "_" + file.getOriginalFilename() (preventing name conflicts), persists Contract with status="UPLOADED", then automatically: (1) runs clauseService.analyzeContract(saved.getId()), (2) runs complianceService.checkCompliance(saved.getId()), (3) updates contract.complianceScore and status="UNDER_REVIEW", saves. Post-upload errors are caught and suppressed (logged to stderr). Saves AuditLog("CONTRACT_UPLOAD", username, "Uploaded contract: " + contractName).
- List<Contract> getAllContracts() — returns all contracts.
- List<Contract> getContractsByUser(String username) — resolves user, returns findByUploadedBy(user).
- Contract getContractById(Long id) — returns contract or throws.
- Contract updateStatus(Long id, String status) — sets status, saves.
- void deleteContract(Long id) — deletes by id.
- boolean isOwner(Long contractId, String username) — returns true if contract.getUploadedBy().getUsername().equals(username).

2. ClauseService

- List<Clause> analyzeContract(Long contractId) throws IOException — resolves contract, constructs file path from app.upload.dir + contract.getFileUrl(). If filename ends with .pdf: uses Apache PDFBox Loader.loadPDF() + PDFTextStripper to extract text; otherwise reads file bytes directly. Runs rule-based clause detection on extracted text using keyword matching (keywords: "payment" → type "Payment Terms" risk LOW; "confidential" → type "Confidentiality" risk LOW; "terminate" → type "Termination" risk MEDIUM; "liability" → type "Limitation of Liability" risk HIGH; "renew" → type "Renewal" risk LOW). Content snippet: 200 chars from keyword index + "...". Risk overrides: content containing "unlimited liability" → HIGH; content containing "penalty" → MEDIUM. Sets contract.status = "UNDER_REVIEW", saves contract. Returns clauseRepository.saveAll(detectedClauses).
- List<Clause> getClausesByContract(Long contractId) — resolves contract, returns clauseRepository.findByContract(contract).
- Clause updateClause(Long id, Clause clauseDetails) — updates clauseType, content, riskLevel, saves.
- void deleteClause(Long id) — *(implied by controller)* deletes clause by id. Controller returns ResponseEntity.ok("Clause deleted successfully").

3. ComplianceService

- ComplianceResult checkCompliance(Long contractId) — resolves contract, loads all clauses via findByContract(). Calculates score: 100 - (highRisk × 15 + mediumRisk × 10 + lowRisk × 5), minimum 0. Builds issues string listing high/medium counts and "MISSING: {clauseType}." for each of the 5 required types not found. Finds or creates ComplianceResult via findByContract(), sets fields, saves.
- List<ComplianceRule> getAllRules() — returns all compliance rules.
- ComplianceResult getResultByContract(Long contractId) — resolves contract, returns findByContract() or throws.

4. AuthService

- JwtAuthenticationResponse login(LoginRequest loginRequest) — authenticates via AuthenticationManager, calls tokenProvider.generateToken(authentication), resolves User by username, saves AuditLog("LOGIN", username, "User logged in successfully"), returns JwtAuthenticationResponse(jwt, user.getUsername(), user.getRole()).
- User register(RegisterRequest registerRequest) — checks username uniqueness throwing RuntimeException("Username already exists"), checks email uniqueness throwing RuntimeException("Email already exists"), encodes password, persists User, saves AuditLog("REGISTER", username, "New user registered with role: " + role), returns saved user.

5. CommentService

- Comment addComment(Long contractId, String content, String username) — resolves contract and reviewer, creates Comment, saves.
- List<Comment> getCommentsByContract(Long contractId) — resolves contract, returns findByContract(contract).

6. ReportService

- List<Report> getAllReports() — returns all reports.
- Report generateContractSummaryReport() — counts total contracts, creates Report(type="Contract Summary", summary="Total contracts in system: {total}"), saves.
- Report generateComplianceOverview() — creates Report(type="Compliance Overview", summary="Aggregated compliance metrics across all reviewed contracts."), saves.
- Report generateRiskAnalysis() — counts HIGH risk clauses across all clauses, creates Report(type="Risk Analysis", summary="Total high risk clauses detected: {count}"), saves.

Controller Layer

Controllers use @RestController and @Autowired field injection. Authentication context accessed via org.springframework.security.core.Authentication parameter.

1. AuthController (/api/auth)

@RestController, @RequestMapping("/api/auth"). No @PreAuthorize — public endpoints.

- **POST** /api/auth/login**:** @PostMapping("/login"). Accepts @RequestBody LoginRequest. Returns ResponseEntity.ok(authService.login(loginRequest)) — 200 OK with JwtAuthenticationResponse.
- **POST** /api/auth/register**:** @PostMapping("/register"). Accepts @RequestBody RegisterRequest. Returns ResponseEntity.ok(authService.register(registerRequest)) — 200 OK with saved User.

2. ContractController (/api/contracts)

Must be @RestController-annotated . Must map to "/api/contracts" . Must be instantiable via no-arg constructor . Must have at least one method annotated with @DeleteMapping .

- **GET** /api/contracts**:** @GetMapping. Authenticated. Role check in method: ROLE_ADMIN or ROLE_LEGAL_REVIEWER → getAllContracts(); else → getContractsByUser(authentication.getName()).
- **GET** /api/contracts/{id}**:** @GetMapping("/{id}"). Authenticated. Ownership or admin/reviewer check; returns 403 if neither.
- **POST** /api/contracts/upload**:** @PostMapping("/upload"). @PreAuthorize("hasAnyRole('LEGAL_REVIEWER', 'CLIENT')") (contains DOMAIN_ROLE = "LEGAL_REVIEWER"). Accepts MultipartFile file + multiple @RequestParam metadata fields. On IOException returns ResponseEntity.status(500).build(). Returns ResponseEntity.ok(contract) — 200 OK.
- **PUT** /api/contracts/{id}**:** @PutMapping("/{id}"). Authenticated. Accepts @RequestParam String status. Returns ResponseEntity.ok(contractService.updateStatus(id, status)) — 200 OK.
- **DELETE** /api/contracts/{id}**:** @DeleteMapping("/{id}"). Authenticated. Returns ResponseEntity.ok().build() — 200 OK. Message: "Contract deleted successfully." (used in frontend and test constant CRUD_DELETE_MSG).

3. ClauseController (/api/clauses)

@RestController, @RequestMapping("/api/clauses"). Must have a @DeleteMapping-annotated method (del must be non-null when found via reflection).

- **GET** /api/clauses**:** @GetMapping. Authenticated. Accepts @RequestParam Long contractId. Role/ownership check (ADMIN, LEGAL_REVIEWER, or owner). Returns ResponseEntity.ok(clauses) or 403.
- **POST** /api/clauses/analyze**:** @PostMapping("/analyze"). Authenticated. Accepts @RequestParam Long contractId. Returns ResponseEntity.ok(clauseService.analyzeContract(contractId)).
- **PUT** /api/clauses/{id}**:** @PutMapping("/{id}"). Authenticated. Returns ResponseEntity.ok(clauseService.updateClause(id, clause)).
- **DELETE** /api/clauses/{id}**:** @DeleteMapping("/{id}"). Authenticated. Calls clauseService.deleteClause(id). Returns ResponseEntity.ok("Clause deleted

successfully") (exact string — the SEC_DELETE_MSG constant is "Clause deleted successfully" without period).

4. ComplianceController (/api/compliance)

@RestController, @RequestMapping("/api/compliance").

- **GET** /api/compliance/rules**:** @GetMapping("/rules"). Authenticated. Returns List<ComplianceRule>.
- **POST** /api/compliance/check**:** @PostMapping("/check"). Authenticated. Accepts @RequestParam Long contractId. Returns ResponseEntity.ok(complianceService.checkCompliance(contractId)).
- **GET** /api/compliance/results/{contractId}**:** @GetMapping("/results/{contractId}"). Authenticated. Role/ownership check. Returns ResponseEntity.ok(...) or 403.

5. CommentController (/api/comments)

@RestController, @RequestMapping("/api/comments").

- **POST** /api/comments**:** @PostMapping. Authenticated. Accepts @RequestParam Long contractId, @RequestBody String content, Authentication authentication. Returns ResponseEntity.ok(commentService.addComment(...)).
- **GET** /api/comments/{contractId}**:** @GetMapping("/{contractId}"). Authenticated. Returns List<Comment>.

6. AdminController (/api/admin)

@RestController, @RequestMapping("/api/admin"). Class-level @PreAuthorize("hasRole('ADMIN')") (class annotation contains "ADMIN").

- **GET** /api/admin/users**:** @GetMapping("/users"). Returns List<User>.
- **DELETE** /api/admin/users/{id}**:** @DeleteMapping("/users/{id}"). Returns ResponseEntity.ok().build().
- **GET** /api/admin/templates**:** @GetMapping("/templates"). Returns List<ClauseTemplate>.
- **POST** /api/admin/templates**:** @PostMapping("/templates"). Returns saved ClauseTemplate.
- **PUT** /api/admin/templates/{id}**:** @PutMapping("/templates/{id}") (AdminController must have at least one @PutMapping-annotated method). Updates name, category, defaultContent, recommendedRiskLevel, saves.
- **DELETE** /api/admin/templates/{id}**:** @DeleteMapping("/templates/{id}"). Returns ResponseEntity.ok().build().
- **GET** /api/admin/rules**:** @GetMapping("/rules"). Returns List<ComplianceRule>.
- **POST** /api/admin/rules**:** @PostMapping("/rules"). Returns saved ComplianceRule.
- **PUT** /api/admin/rules/{id}**:** @PutMapping("/rules/{id}"). Updates ruleName, description, severityLevel, saves.
- **DELETE** /api/admin/rules/{id}**:** @DeleteMapping("/rules/{id}"). Returns ResponseEntity.ok().build().

- **GET** /api/admin/activity**:** @GetMapping("/activity"). Returns auditLogRepository.findAllByOrderByLogTimestampDesc().

7. ReportController (/api/reports)

@RestController, @RequestMapping("/api/reports"). Authenticated.

- **GET** /api/reports/contracts**:** @GetMapping("/contracts"). Returns reportService.generateContractSummaryReport().
- **GET** /api/reports/compliance**:** @GetMapping("/compliance"). Returns reportService.generateComplianceOverview().
- **GET** /api/reports/risks**:** @GetMapping("/risks"). Returns reportService.generateRiskAnalysis().
- **GET** /api/reports/all**:** @GetMapping("/all"). Returns List<Report>.

Security Implementation

- **JWT signing algorithm:** HS256
- **Security utility class name:** JwtTokenProvider (located in com.example.demo.security.JwtTokenProvider) — full class path: com.example.demo.security.JwtTokenProvider (used as JWT_CLASS_PATH in tests).
- **JWT secret field:** jwtSecret — must be declared private (field name lowercased contains "secret"), injected via @Value("${app.jwt.secret}").
- **JWT expiration field:** jwtExpirationInMs — int, injected via @Value("${app.jwt.expiration}").

Exact method names in JwtTokenProvider.java**:**

- generateToken(Authentication authentication) — builds HS256 JWT using authentication.getName() as subject, returns String (must be invokable as clazz.getDeclaredMethod("generateToken", Authentication.class), token must have 3 parts split by ".").
- validateToken(String authToken) — parses JWT, returns true on valid token, false on JwtException or IllegalArgumentException (clazz.getDeclaredMethod("validateToken", String.class), round-trip must return true).
- getUsernameFromJWT(String token) — parses JWT claims subject, returns String username (clazz.getDeclaredMethod("getUsernameFromJWT", String.class), extracted username must equal AUTH_EMAIL = "admin@legallens.com").

Note: JwtTokenProvider must be instantiable via **no-arg constructor** (tests call clazz.getDeclaredConstructor().newInstance() then set fields via reflection). Fields jwtSecret and jwtExpirationInMs are set by reflection using setField(provider, "jwtSecret", "PeakPerform2024SuperSecretKeyForHmacSHA256SigningAtLeast256BitsLong") and setField(provider, "jwtExpirationInMs", 3600000).

SecurityConfig.java (located in com.example.demo.security.SecurityConfig):

- Must be annotated with @Configuration , @EnableWebSecurity, @EnableMethodSecurity.
- Must declare a @Bean method returning SecurityFilterChain — its getSimpleName() must contain "SecurityFilterChain" .
- Must declare a @Bean method returning PasswordEncoder — its getSimpleName() must contain "PasswordEncoder" .
- Must declare a method whose name (lowercased) contains "cors" .
- Session management: STATELESS.
- JwtAuthenticationFilter added before UsernamePasswordAuthenticationFilter.

CORS policy (corsFilter() bean):

- config.setAllowCredentials(true)
- config.addAllowedOriginPattern("*")
- config.addAllowedHeader("*")
- config.addAllowedMethod("*")
- Registered on "/**"

Path authorisation rules:

- /api/auth/** → permitAll()
- /api/contracts/** → authenticated()
- /api/clauses/** → authenticated()
- /api/compliance/** → authenticated()
- /api/comments/** → authenticated()
- /api/reports/** → authenticated()
- Any other request → authenticated()
- Method-level @PreAuthorize handles fine-grained role enforcement.

UserDetailsServiceImpl:

- Implements UserDetailsService. Loads user by username; throws UsernameNotFoundException("User not found with username: " + username) if not found.
- Returns Spring Security User with single SimpleGrantedAuthority(user.getRole()).

JwtAuthenticationFilter:

- Extends OncePerRequestFilter. Parses Authorization: Bearer <token> header. On valid token: calls getUsernameFromJWT(), loads UserDetails, sets UsernamePasswordAuthenticationToken in SecurityContextHolder.

Exception Handling

Required Package: com.example.demo.exception

The central handler is GlobalExceptionHandler. Must be annotated with @ControllerAdvice .

ResourceNotFoundException:

- Must exist at com.example.demo.exception.ResourceNotFoundException .
- Annotated with @ResponseStatus(HttpStatus.NOT_FOUND).
- Constructor: ResourceNotFoundException(String message).

BusinessValidationException:

- Must exist at com.example.demo.exception.BusinessValidationException .
- Annotated with @ResponseStatus(HttpStatus.CONFLICT).
- Constructor: BusinessValidationException(String message).

GlobalExceptionHandler handlers:

- @ExceptionHandler(RuntimeException.class) — returns ResponseEntity with HTTP **400 BAD_REQUEST** and Map<String, String> body { "error": ex.getMessage() }.
- @ExceptionHandler(Exception.class) — returns ResponseEntity with HTTP **500 INTERNAL_SERVER_ERROR** and Map<String, String> body { "error": "An internal server error occurred", "message": ex.getMessage() }.

Data Transfer Objects (DTOs)
LoginRequest

Fields: username (String, @NotBlank(message="Username is required")), password (String, @NotBlank(message="Password is required"))

Note: Must have at least one field annotated with @NotBlank .

RegisterRequest

Fields: username (String), password (String), email (String), fullName (String), role (String — ROLE_CLIENT, ROLE_LEGAL_REVIEWER, ROLE_ADMIN)

JwtAuthenticationResponse

Fields: accessToken (String), tokenType (String, default "Bearer"), username (String), role (String)

Constructor: JwtAuthenticationResponse(String accessToken, String username, String role).

API Endpoints

Method	Endpoint	Role	Description
POST	/api/auth/login	PUBLIC	Authenticate user. Returns JwtAuthenticationResponse with accessToken, username, role.
POST	/api/auth/register	PUBLIC	Register new user. Returns saved User.

			Throws 400 on duplicate username/email.
GET	/api/contracts	AUTHENTICATED	Admin/Reviewer: all contracts. Client: own contracts.
GET	/api/contracts/{id}	AUTHENTICATED	Get single contract. 403 if not owner/admin/reviewer.
POST	/api/contracts/upload	LEGAL_REVIEWER, CLIENT	Upload contract file + metadata. Auto-runs clause analysis and compliance check. Returns 200 with saved Contract.
PUT	/api/contracts/{id}	AUTHENTICATED	Update contract status via ?status= query param. Returns 200.
DELETE	/api/contracts/{id}	AUTHENTICATED	Delete contract. Returns 200 OK.
GET	/api/clauses	AUTHENTICATED	Get clauses for a contract (?contractId=). 403 if not owner/admin/reviewer.
POST	/api/clauses/analyze	AUTHENTICATED	Trigger clause analysis for a contract (?contractId=). Returns list of detected clauses.
PUT	/api/clauses/{id}	AUTHENTICATED	Update clause type, content, riskLevel. Returns updated Clause.
DELETE	/api/clauses/{id}	AUTHENTICATED	Delete clause. Returns 200 OK with body "Clause deleted successfully".
GET	/api/compliance/rules	AUTHENTICATED	Returns all ComplianceRule entities.
POST	/api/compliance/check	AUTHENTICATED	Run compliance check for contract (?contractId=). Returns ComplianceResult.
GET	/api/compliance/results/{contractId}	AUTHENTICATED	Get compliance result. 403 if not owner/admin/reviewer.
POST	/api/comments	AUTHENTICATED	Add comment to contract (?contractId=). Returns saved Comment.
GET	/api/comments/{contractId}	AUTHENTICATED	Get all comments for a contract.
GET	/api/reports/contracts	AUTHENTICATED	Generate and return contract summary report.
GET	/api/reports/compliance	AUTHENTICATED	Generate and return compliance overview report.

GET	/api/reports/risks	AUTHENTICATED	Generate and return risk analysis report.
GET	/api/reports/all	AUTHENTICATED	Returns all saved reports.
GET	/api/admin/users	ADMIN	Returns all users.
DELETE	/api/admin/users/{id}	ADMIN	Delete user. Returns 200 OK.
GET	/api/admin/templates	ADMIN	Returns all clause templates.
POST	/api/admin/templates	ADMIN	Create clause template. Returns saved ClauseTemplate.
PUT	/api/admin/templates/{id}	ADMIN	Update clause template. Returns updated ClauseTemplate.
DELETE	/api/admin/templates/{id}	ADMIN	Delete clause template. Returns 200 OK.
GET	/api/admin/rules	ADMIN	Returns all compliance rules.
POST	/api/admin/rules	ADMIN	Create compliance rule. Returns saved ComplianceRule.
PUT	/api/admin/rules/{id}	ADMIN	Update compliance rule. Returns updated ComplianceRule.
DELETE	/api/admin/rules/{id}	ADMIN	Delete compliance rule. Returns 200 OK.
GET	/api/admin/activity	ADMIN	Returns all audit logs ordered by logTimestamp DESC.

Frontend Page Specifications
1. pages/LoginPage.js

Purpose: Authentication entry point using Ant Design Form.

Implementation Requirements:

- Renders "LegalLens" title and subtitle "Contract Clause & Compliance Analyzer".
- Uses Ant Design Form with name="login" and onFinish={onFinish}.
- Username field: Form.Item name="username" containing <Input placeholder="Username" prefix={<UserOutlined />}>.
- Password field: Form.Item name="password" containing <Input.Password placeholder="Password">.
- Submit button with text "Log in" .
- onFinish(values): calls authService.login(values). On success: dispatches loginSuccess({ user: { username: data.username, role: data.role }, token: data.accessToken }), shows message.success('Login successful!'), navigates to '/'.
- On login success: localStorage.setItem('token', data.accessToken) and localStorage.setItem('user', JSON.stringify({ username, role })) are set via loginSuccess reducer .
- On error: message.error(error.response?.data?.error || 'Login failed').

- Link to /register with text "register now!".

Additional Requirements:

Login Success Flow:

- On successful login: authService.login returns JwtAuthenticationResponse with accessToken, username, role

- Dispatches loginSuccess action with payload: { user: { username, role }, token: accessToken }

- loginSuccess reducer must:

  - Set state.user = payload.user

  - Set state.token = payload.token

  - Set state.isAuthenticated = true

  - Call localStorage.setItem('token', payload.token)

  - Call localStorage.setItem('user', JSON.stringify(payload.user))

- After dispatch: shows message.success('Login successful!')

- Navigates to '/' (Dashboard)

Login Failure Flow:

- On 401 Unauthorized: displays message.error(error.response?.data?.error || 'Login failed')

- On 400 Bad Request: displays validation error message

- Form must remain usable after error

- Password field must be cleared or allow retry

Username Field:

- Must have placeholder text "Username" (exact text)

- Must be an Input component with prefix={<UserOutlined />}

Password Field:

- Must be Input.Password component with placeholder "Password"

2. pages/RegisterPage.js

Purpose: New user self-registration.

Implementation Requirements:

- Renders "Join LegalLens" title.
- Uses Ant Design Form with name="register".
- Fields: fullName (required), username (required), email (required, type email), password (required), role (Select with options ROLE_CLIENT, ROLE_LEGAL_REVIEWER, ROLE_ADMIN).
- Submit button text: "Register".
- On success: message.success('Registration successful! Please login.'), navigates to '/login'.
- On error: message.error(error.response?.data?.error || 'Registration failed').

3. layout/MainLayout.js

Purpose: Role-aware application shell using Ant Design Layout.

Implementation Requirements:

- Uses Ant Design Layout, Sider, Header, Content, Menu.
- **Sider** (theme="dark") — rendered as <aside className="dark"> in tests .
- Brand text in sider: "LegalLens" (when not collapsed), "LL" (when collapsed).
- Sidebar menu items always present: "Dashboard" (key "/").
- Additional menu items always present: "Compliance" (key "/compliance") , "Reports" (key "/reports") .
- When user?.role === 'ROLE_LEGAL_REVIEWER' or 'ROLE_CLIENT': "Upload Contract" item (key "/upload") (getByText(/Upload Contract/i) must be in document; note setupTests mock useSelector returns ROLE_ADMIN, so "Upload Contract" is hidden for admin).
- When user?.role === 'ROLE_ADMIN': "Admin Panel" item (key "/admin") checks getByText(/Management/i) is in document. Note: sidebar menu label must contain text matching /Management/i — "Admin Panel" or an item labeled "Management" satisfies this (setupTests MockMainLayout renders items for ROLE_ADMIN).
- **Header** displays: "Logged in as admin" text .
- Toggle button in header for sidebar collapse.
- **Logout button** text "Logout": dispatches logout() redux action, calls localStorage.removeItem('token'), localStorage.removeItem('user'), navigates to '/login').
- Content area renders {children} .
- Menu onClick: navigates to item key.

Additional Requirements:

Sidebar Toggle Button:

- Header must contain a toggle button (MenuUnfoldOutlined / MenuFoldOutlined icon)

- Clicking toggle button must collapse/expand the Sider component

- Sider width must change from 200px to 80px when collapsed

- When collapsed: brand text changes from "LegalLens" to "LL"

Content Area:

- The <Content> component must render {children} props passed to MainLayout

- Must support nested routing via React Router Outlet or children prop

Logout Behavior:

- Logout button click must dispatch logout() Redux action

- After logout dispatch: must call localStorage.removeItem('token') and localStorage.removeItem('user')

- Redux state.auth.user must become null

- Redux state.auth.token must become null

- Must navigate to '/login' using useNavigate()

Header Display:

- Must display text "Logged in as {username}" where username comes from Redux state.auth.user.username

- Example: "Logged in as admin" for admin user

4. pages/Dashboard.js

Purpose: Main landing page with contract list and role-specific statistics.

Implementation Requirements:

- Page title: "{role_without_prefix} Dashboard" rendered via user?.role?.replace('ROLE_', '').replace('_', ' ') — the text "Contract Compliance Dashboard" must appear in document.
- On mount via useEffect: calls contractService.getContracts().
- Role-specific stat cards:
    - ROLE_ADMIN: Total Contracts, Approved, Rejected, Avg Compliance (averageFocusScore %).
    - ROLE_LEGAL_REVIEWER: Pending Review, High Risk, Recent Uploads.
    - ROLE_CLIENT: My Contracts, Approved Docs.

- Ant Design Table with columns: contractName, contractType (Tag), status (coloured Tag), complianceScore (Progress bar), Action (View button navigating to /contract/{id}).
- Error handling: message.error('Failed to fetch contracts') on failure.

Additional Requirements:

Page Heading:

- Must display heading text "Contract Compliance Dashboard" exactly (case-sensitive)

Data Fetching:

- On component mount via useEffect: calls contractService.getContracts()

- While loading: displays Ant Design Spin component

- On successful fetch: stores response data in local state and displays in Table

Error Handling:

- On 401 Unauthorized response: displays message.error('Session expired. Please login again.')

- On 500 Internal Server Error: displays message.error('Failed to fetch contracts')

- Error must not crash the component

Table Columns:

- Must render Ant Design Table with columns: contractName, contractType (as Tag), status (coloured Tag), complianceScore (as Progress bar), Action (View button)

- View button navigates to `/contract/{id}` route

5. pages/UploadPage.js

Purpose: Contract upload form with full metadata.

Implementation Requirements:

- Page title: "Upload New Contract" and subtitle "Provide complete contract details...".
- Must render section heading "Basic Information" via <Card title={<InfoCircleOutlined /> Basic Information}> .
- Must render page heading "Upload and Analyze Contract" .
- Form fields: contractName (required), contractType (Select — options: Employment Contract, NDA, Lease Agreement, Vendor Agreement, Service Agreement, Partnership Agreement), description, firstParty, secondParty, startDate (DatePicker), endDate (DatePicker), contractValue (InputNumber,

default 0), jurisdiction (Select — options: United States, United Kingdom, EU, India, Singapore, Other), file upload.
- On submit: calls contractService.uploadContract(values) with FormData. On success: message.success('Contract uploaded and analysis started successfully!'), navigates to '/'. On error: message.error('Failed to upload contract').

Additional Requirements:

Form Validation:

- All required fields (contractName, file) must show validation warning when empty

- Ant Design Form.Item rules must include required: true for mandatory fields

- On validation failure: form does not submit, shows error messages under each field

Success Handling:

- On successful upload (201 response): displays message.success('Contract uploaded and analysis started successfully!')

- After success message: navigates to '/' (Dashboard) using navigate()

- Success message must persist for 3 seconds

Error Handling:

- On 500 Internal Server Error: displays message.error('Failed to upload contract')

- On network error: displays message.error('Network error. Please try again.')

- Error must not crash the component

- Form must remain usable after error

File Upload Format:

- Uses FormData to append file and all metadata fields

- Content-Type header must be multipart/form-data

- Date fields (startDate, endDate) formatted as YYYY-MM-DD strings

6. pages/ContractDetails.js

Purpose: Full view of a single contract with clause listing, compliance info, and comments.

Implementation Requirements:

- Page heading: "Contract Details" .

- On mount via useEffect([id]): calls contractService.getContract(id), clauseService.getClauses(id), commentService.getComments(id) in parallel.
- Shows contract metadata, clauses list with risk badges, compliance score Progress bar.
- "Analyze" button: calls clauseService.analyzeContract(id), refreshes data, shows message.success('Analysis complete!').
- Clause edit modal: Form with clauseType, content, riskLevel fields. Update calls clauseService.updateClause(id, values).
- Comment section: textarea newComment, submit adds via commentService.addComment(id, content, username).
- Status update (LEGAL_REVIEWER/ADMIN): calls contractService.updateStatus(id, status).
- Error handling: message.error('Failed to fetch details').

7. pages/AdminDashboard.js

Purpose: Admin control panel for users, templates, rules, activity.

Implementation Requirements:

- Page heading: "Admin Control Panel" .
- Ant Design Tabs with four tabs: users, templates, rules, activity.
- On tab change loadTabData(tab): loads data from adminService.getUsers() / getTemplates() / getRules() / getActivityLogs().
- Users tab: table with Delete button calling adminService.deleteUser(id).
- Templates tab: CRUD — Add/Edit via modal form (name, category, defaultContent, recommendedRiskLevel), adminService.saveTemplate(template). Delete calls adminService.deleteTemplate(id). Satisfies UPDATE verification .
- Rules tab: CRUD — Add/Edit via modal form (ruleName, description, severityLevel), adminService.saveRule(rule). Delete calls adminService.deleteRule(id).
- Activity tab: table showing audit logs (action, username, details, logTimestamp).

8. pages/CompliancePage.js

Purpose: Overview of contracts and their compliance scores, with link to detailed check.

Implementation Requirements:

- On mount: fetches contractService.getContracts().
- Table of contracts with compliance scores and link to /compliance-check/{id}.

9. pages/ComplianceDashboard.js

Purpose: Detailed compliance result for a specific contract.

Implementation Requirements:

- Reads id from URL via useParams().

- On mount: calls complianceService.getResults(id). On failure: calls complianceService.checkCompliance(id).
- Displays complianceScore as Progress bar, issuesFound as list items, rule list.
- "Run Check" button: calls complianceService.checkCompliance(id).

10. pages/ReportsPage.js

Purpose: Summary reports on contract data.

Implementation Requirements:

- On mount: calls contractService.getContracts().
- Displays report tables: by type distribution, compliance score distribution, risk level breakdown.
- "Download / Print" button: calls window.print().

Frontend Service Configurations (Axios API)
services/api.js

Purpose: Axios instance configuration.

Implementation Requirements:

- baseURL: 'http://localhost:8080/api'
- Request interceptor: reads localStorage.getItem('token') and injects Authorization: Bearer {token} header if present.
- Default export is the Axios instance (used by all other services via import api from './api').

services/authService.js

- login(credentials) — POST /auth/login, returns response.data (JwtAuthenticationResponse)
- register(userData) — POST /auth/register, returns response.data
- Default export: { login, register }

services/contractService.js

- getContracts() — GET /contracts
- getContract(id) — GET /contracts/{id}
- uploadContract(values) — POST /contracts/upload with FormData (Content-Type: multipart/form-data). Extracts file from values.file[0].originFileObj || values.file[0]. Date fields formatted via .format('YYYY-MM-DD').
- updateStatus(id, status) — PUT /contracts/{id}?status={status}
- deleteContract(id) — DELETE /contracts/{id}
- Default export: { getContracts, getContract, uploadContract, updateStatus, deleteContract }

services/clauseService.js

- analyzeContract(contractId) — POST /clauses/analyze?contractId={contractId}
- getClauses(contractId) — GET /clauses?contractId={contractId}
- updateClause(id, clause) — PUT /clauses/{id} with clause body
- Default export: { analyzeContract, getClauses, updateClause }

services/commentService.js

- addComment(contractId, content) — POST /comments?contractId={contractId} with content body
- getComments(contractId) — GET /comments/{contractId}

services/complianceService.js

- checkCompliance(contractId) — POST /compliance/check?contractId={contractId}
- getResults(contractId) — GET /compliance/results/{contractId}
- getRules() — GET /compliance/rules
- Default export: { checkCompliance, getResults, getRules }

services/reportService.js

- getContractReport() — GET /reports/contracts
- getComplianceReport() — GET /reports/compliance
- getRiskReport() — GET /reports/risks
- getAllReports() — GET /reports/all
- Default export: { getContractReport, getComplianceReport, getRiskReport, getAllReports }

services/adminService.js

- getUsers() — GET /admin/users
- deleteUser(id) — DELETE /admin/users/{id}
- getTemplates() — GET /admin/templates
- saveTemplate(template) — if template.id: PUT /admin/templates/{id}; else: POST /admin/templates
- deleteTemplate(id) — DELETE /admin/templates/{id}
- getRules() — GET /admin/rules
- saveRule(rule) — if rule.id: PUT /admin/rules/{id}; else: POST /admin/rules
- deleteRule(id) — DELETE /admin/rules/{id}
- getActivityLogs() — GET /admin/activity
- Default export: { getUsers, deleteUser, getTemplates, saveTemplate, deleteTemplate, getRules, saveRule, deleteRule, getActivityLogs }

Additional Requirements:

getContracts Method:

- Must send GET request to contracts endpoint
- Must return a Promise that resolves to an array of contract objects
- Each contract object in the array must contain contract name field
- Each contract object must contain contract type field
- Each contract object must contain status field
- Each contract object must contain compliance score field

deleteContract Method:

- Must send DELETE request to specific contract endpoint using the contract identifier
- Must return a Promise that resolves on successful deletion
- Successful deletion response must have success status code
- Response body must contain exact message confirming contract deletion

Response Status Expectations:

- Successful data fetch must return success status code
- Unauthorized request must return authentication error status code
- Server error must return internal server error status code
- Deletion of non-existent contract must return not found status code

Frontend Store Configurations (Redux Toolkit)
store/index.js

Purpose: Root store.

Implementation Requirements:

- Configures store via configureStore with single reducer { auth: authReducer }.
- Default export: store.
- store.getState() must expose state.auth .

store/slices/authSlice.js

Purpose: Authentication session state.

Initial state: { user: JSON.parse(localStorage.getItem('user')) || null, token: localStorage.getItem('token') || null, isAuthenticated: !!localStorage.getItem('token') }.

- **Action** loginSuccess(payload)**:** Sets state.user = payload.user, state.token = payload.token, state.isAuthenticated = true. Persists localStorage.setItem('user', JSON.stringify(payload.user)) and localStorage.setItem('token', payload.token) JSON.parse(localStorage.getItem('user')).role equals 'ROLE_ADMIN'; **Action** logout**:** Sets user = null, token = null, isAuthenticated = false. Calls

localStorage.removeItem('user') and localStorage.removeItem('token') is null; store.getState().auth.user is null after { type: 'auth/logout' }).
- Exports: { loginSuccess, logout } as named action creators; default export is reducer.

Additional Requirements:

loginSuccess Action Behavior:

- Must store user object and token in Redux state when login succeeds
- Must set isAuthenticated flag to true after successful login
- Must automatically save token to localStorage when loginSuccess is dispatched
- Must automatically save user object (as JSON string) to localStorage when loginSuccess is dispatched
- User object stored in localStorage must contain at minimum username and role fields

logout Action Behavior:

- Must set user to null in Redux state when logout is dispatched
- Must set token to null in Redux state when logout is dispatched
- Must set isAuthenticated flag to false in Redux state
- Must remove token key from localStorage when logout is dispatched
- Must remove user key from localStorage when logout is dispatched

Initial State Requirements:

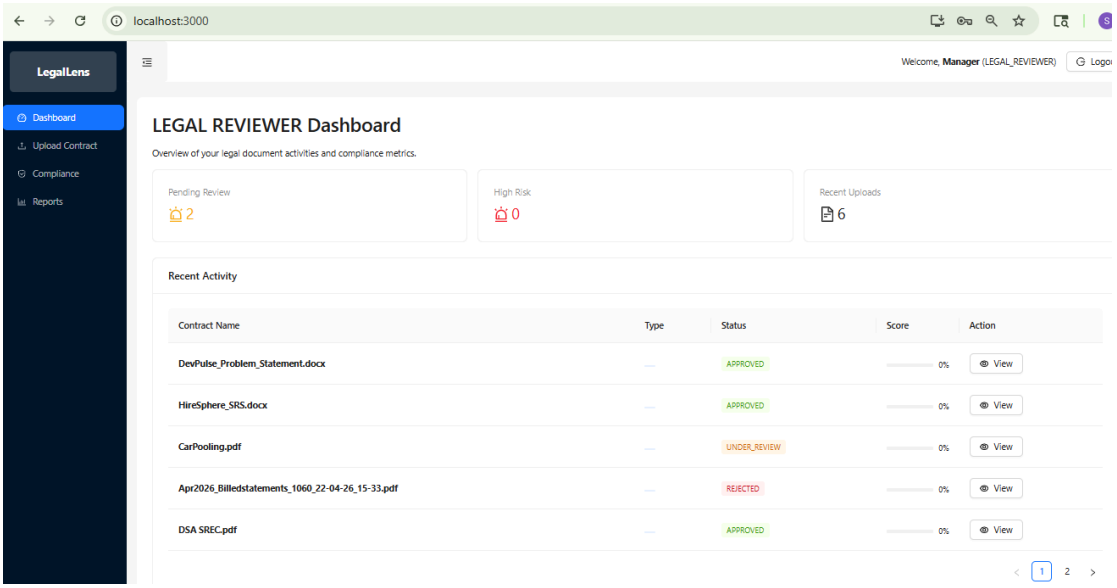
- Must read token from localStorage during initial state creation
- Must read user from localStorage during initial state creation (parsed from JSON)
- Must set isAuthenticated based on whether token exists in localStorage
- If token exists in localStorage, isAuthenticated must be true
- If token does not exist in localStorage, isAuthenticated must be false

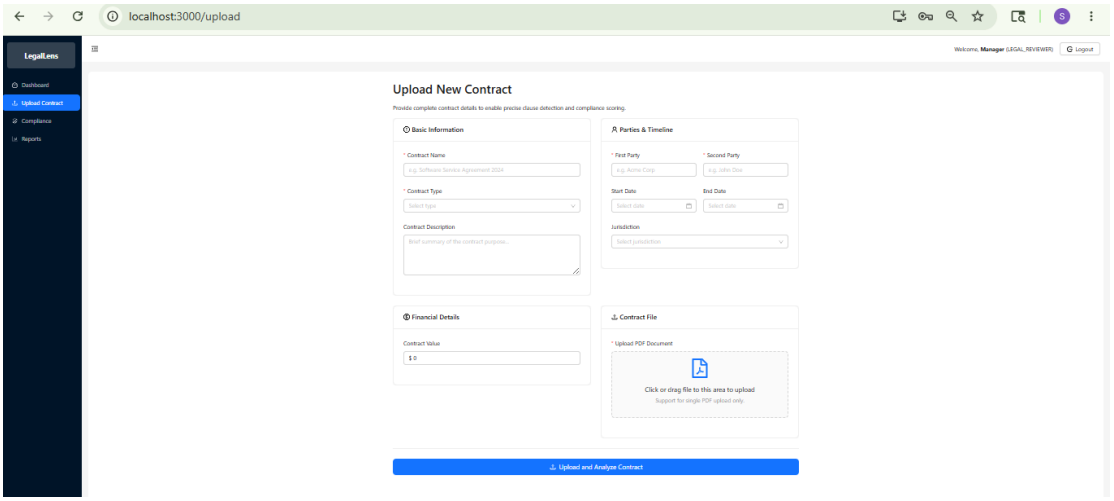
Test Validation Points:

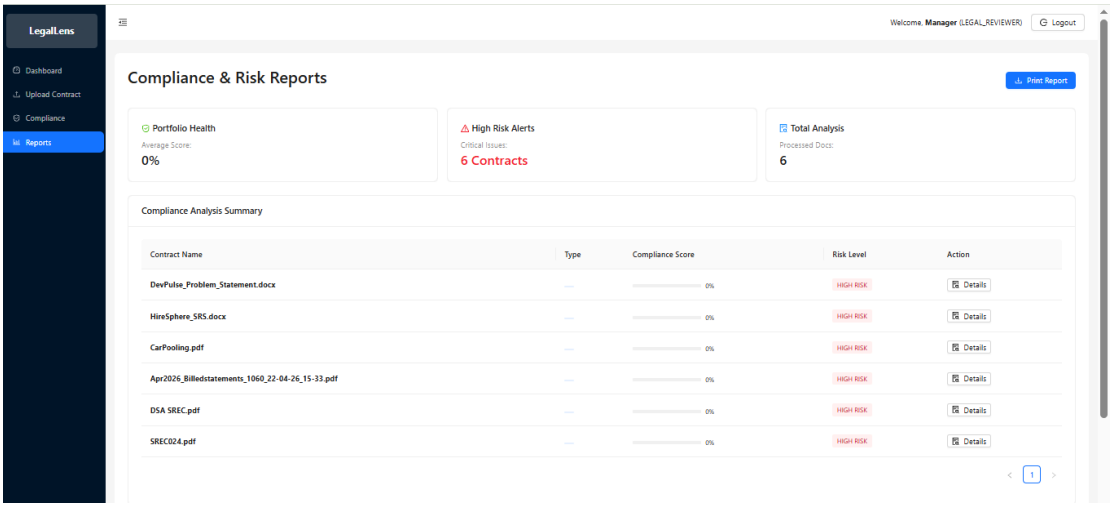
- After loginSuccess dispatch, localStorage token must match the token from action payload
- After loginSuccess dispatch, stored user role must match the role from action payload

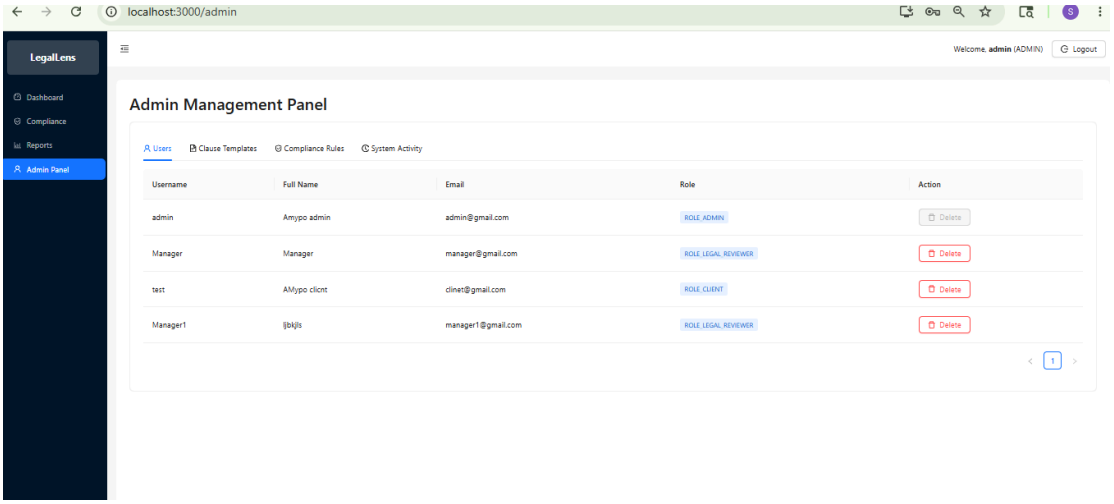
- After logout dispatch, localStorage token must be completely removed
- After logout dispatch, Redux auth user must be null

Demo User Interface









End of Documentation